

无哈希函数下的哈希表设计综述

巨文博

(南京大学智能软件与工程学院 苏州 215000)

摘 要 哈希表是字典问题中最常用的随机化数据结构之一,通常依赖哈希函数将键映射到存储位置。然而,当希望获得极低失败概率时,传统构造会受到显式哈希函数族能力的限制。Kuszmaul 在论文“A Hash Table Without Hash Functions, and How to Get the Most Out of Your Random Bits”中提出了一种不同思路:不再直接使用哈希函数,而是将随机性嵌入到数据结构内部。本文围绕该论文的主要结构展开综述,重点介绍旋转基数树的基本思想、增强旋转基数树降低失败概率的方法、低随机性旋转基数树压缩随机比特数量的思路,以及相关结构在空间效率上的意义。

关键词 哈希表; 随机化字典; 旋转基数树; 失败概率; 随机比特; 简洁字典

1 引言

字典(dictionary)是一类支持插入、删除和查询操作的数据结构。在算法设计中,随机化字典通常也被称为哈希表。经典哈希表通过哈希函数将键映射到桶或槽位中,从而在期望意义或高概率意义下实现常数时间操作。

本文综述的对象是 Kuszmaul 提出的无哈希函数哈希表构造^[1]。该论文关注的问题是:在存储 n 个 $\Theta(\log n)$ 位键值对、使用线性空间并支持常数时间操作的前提下,哈希表能够提供多强的概率保证。

传统研究通常将概率保证与哈希函数族联系在一起。如果能够使用完全随机哈希函数,则可以得到非常强的负载均衡效果;但完全随机函数难以显式高效实现。另一方面,已知的常数时间显式哈希函数族往往只能提供有限独立性,从而限制了失败概率的进一步降低。

Kuszmaul 的核心思路是绕开这一瓶颈:构造一种具有哈希表功能的随机化字典,但不再直接使用哈希函数。随机性不表现为 $h(x)$ 这样的键到桶的映射,而是以节点数组随机旋转的形式嵌入到数据结构内部。

2 研究背景

2.1 字典与失败概率

一个字典需要维护一个动态集合,并支持查询某个键是否存在以及取回对应值。在随机化字典中,所谓失败概率通常指某次操作不能在常数时间内完成的概率。当失败概率不超过 $1/\text{poly}(n)$ 时,通常称该结构以高概率成功。

确定性的常数时间字典是一个重要问题。已有工作如动态融合节点(dynamic fusion node)可以对较小规模集合实现

确定性常数时间操作^[2],但对于 $\Theta(n)$ 规模的一般集合,确定性常数时间字典仍然是困难问题。

2.2 哈希函数瓶颈

过去关于高概率哈希表的研究大多依赖哈希函数族。问题在于,如果使用完全随机哈希函数,理论上可以得到很强的负载均衡;但若将其替换为可显式构造、常数时间求值的哈希函数族,概率保证往往会下降。

因此,论文提出的方向不是继续寻找更强的传统哈希函数,而是改变随机性的使用方式:将随机性直接放入数据结构本身。这一点是本文后续结构的核心出发点。

3 旋转基数树(Rotated Radix Trie)

3.1 基本结构

旋转基数树(rotated radix trie)是论文中的基础结构。普通基数树可以根据键的不同片段逐层选择子节点,但如果直接为每个内部节点维护一个大数组,整体空间可能达到二次量级。

旋转基数树的做法是:为每个内部节点 s 选择一个随机旋转变量 r_s ,然后把各个节点数组经过循环旋转后叠加到一个总数组中。如果某个非空指针原本位于子节点编号 c ,则旋转后映射到

$$\phi(s, c) = (c + r_s) \bmod n. \quad (1)$$

这个公式体现了该结构与传统哈希表的区别:它没有对每个键计算哈希值,而是对节点数组整体进行随机旋转。

3.2 球、桶与负载

为了分析该结构, 论文将每个非空指针称为一个球 (ball), 将叠加后总数组中的位置称为桶 (bin)。一个球可以表示为 (s, c) , 其中 s 是源节点, c 是该节点中的子节点编号。

对某个桶 j , 设 Y_j 表示映射到该桶的球数量, 则 Y_j 就是该桶的负载 (load)。由于总共有 $O(n)$ 个球, 而每个球落到固定桶的概率约为 $1/n$, 因此可以得到

$$\mathbb{E}[Y_j] = O(1). \quad (2)$$

论文进一步使用 Chernoff 界说明 Y_j 超过 $\text{polylog}(n)$ 的概率很小。只要每个桶的负载被控制在 $\text{polylog}(n)$ 以内, 就可以用动态融合节点存储每个桶内部的小集合, 从而保持常数时间操作。

3.3 结构小结

旋转基数树的意义在于, 它证明了可以通过随机旋转数组而不是直接使用哈希函数来构造随机化字典。它本身已经达到线性空间和高概率常数时间操作, 但失败概率还不够强, 因此还需要增强旋转基数树进一步放大概率保证。

4 增强旋转基数树 (Amplified Rotated Trie)

4.1 溢出球 (Overflow Ball)

增强旋转基数树 (amplified rotated trie) 是旋转基数树的改进版本, 目标是将失败概率降低到接近最优的量级。该结构首先允许主结构中出现少量溢出。当某个桶中已经存放了 $\text{polylog}(n)$ 个球, 而新的球仍然映射到该桶时, 这个新的球被称为溢出球 (overflow ball)。

溢出球不再直接放入原桶, 而是存储到辅助结构 Q 中。这样主结构中每个桶的规模仍然可以被控制住。

4.2 控制溢出数量

辅助结构 Q 本身并不一定空间高效, 因此必须证明溢出球的总数 q 足够小。论文将 q 看成所有随机旋转变量的函数:

$$f(r_1, r_2, \dots, r_m) = q. \quad (3)$$

为了分析该函数的集中性, 论文使用了 McDiarmid 不等式^[3]。直观上, 如果只改变某一个节点的随机旋转变量, 最多只会影响该节点产生的球。原论文第 IV 节将分支因子 (fanout) 调整为 n^δ , 使得单个随机变量对 q 的影响被限制在 n^δ 范围内, 从而满足使用 McDiarmid 不等式所需的有界差分条件。

4.3 概率保证

通过降低分支因子、引入溢出球, 并对溢出球总数进行集中性分析, 增强旋转基数树可以将失败概率降低到

$$O\left(\frac{1}{n^{1-\epsilon}}\right) \quad (4)$$

的量级。这个结果显著强于基础旋转基数树, 也接近理论上可以期待的最强概率保证。

5 低随机性旋转基数树 (Budget Rotated Trie)

低随机性旋转基数树 (budget rotated trie) 是论文的另一方向。与增强旋转基数树追求极低失败概率不同, 低随机性旋转基数树更关注随机比特数量, 希望在保持 $1/\text{poly}(n)$ 失败概率的同时, 只使用约 $O(\log n \log \log n)$ 个随机比特。

为了减少随机比特的使用, 低随机性旋转基数树不再为大量内部节点直接生成完全独立的随机旋转变量。一个基本做法是减少真正需要显式创建的内部节点数量: 当某个子树规模较小时, 可以先用动态融合节点直接存储, 只有当子树规模超过阈值后才创建新的内部节点。

在此基础上, 论文进一步改变球到桶的映射方式。基础旋转基数树使用 $\phi(s, c) = (c + r_s) \bmod n$, 而低随机性旋转基数树将全部桶分为 n^δ 个组, 每组大小为 $n^{1-\delta}$, 并使用

$$\psi(s, c) = ((c + a_s) \bmod n^\delta) \cdot n^{1-\delta} + b_s \quad (5)$$

进行映射。其中 a_s 用来决定球被分配到哪个组, b_s 用来决定它在组内的具体位置。这样的拆分使得两类随机参数可以用不同方法生成: a_s 可由常数独立哈希函数产生, 用于控制各组中的球数量; b_s 则由具有负载均衡性质的逐渐增强独立性哈希函数 (gradually-increasing-independence hash functions) 产生, 用于控制组内最大负载。

逐渐增强独立性哈希函数的直接求值时间并非常数, 但在低随机性旋转基数树中, 它们主要作为生成随机参数的工具。由于新的内部节点只会对应子树规模达到 $\text{polylog}(n)$ 后创建, 相关初始化成本可以被摊销到之前的插入过程中。这样一来, 结构仍然能够维持高概率常数时间操作, 同时显著减少随机比特使用量。

综合各步构造可得, 低随机性旋转基数树是一个随机化线性空间字典, 可以使用 $O(\log n \log \log n)$ 个随机比特存储最多 n 个 $\Theta(\log n)$ 位的键值对, 每次操作以 $1 - 1/\text{poly}(n)$ 的概率在常数时间内完成。

6 空间效率

除了操作时间和失败概率外, 空间效率也是字典结构中的重要指标。对于从全集 U 中存储 n 个键的集合, 任意字典都需要至少

$$B(|U|, n) = \log \binom{|U|}{n} \quad (6)$$

比特的信息量。因此，简洁字典 (succinct dictionary) 的目标是将空间控制在接近该信息论下界的范围内，通常写作 $(1 + o(1))B(|U|, n)$ 。

Kuszmaul 的论文中进一步指出，旋转树相关结构可以通过黑盒转换得到简洁字典，并且基本保留原结构的概率保证。该转换与 Raman 和 Rao 的简洁动态字典工作有关^[4]，其中低随机性旋转基数树也作为关键组件出现。

从综述角度看，这一部分说明论文的贡献并不只体现在失败概率上。增强旋转基数树强调极低失败概率，低随机性旋转基数树强调随机比特节省，而简洁化转换则进一步说明这些结构有机会接近最优空间使用。

7 结构关系与讨论

综合来看，论文中的几个结构可以理解为围绕旋转基数树展开的三个方向。

首先，旋转基数树是基础结构。它通过随机旋转节点数组，将传统哈希函数中的随机性转移到数据结构内部，说明“不直接使用哈希函数”仍然可以实现随机化字典。

其次，增强旋转基数树关注失败概率。它在基础结构上引入溢出球和辅助结构，并通过有界差分方法控制溢出规模，从而获得远强于普通高概率保证的失败概率。

最后，低随机性旋转基数树关注随机比特数量。它通过减少内部节点、改变映射方式以及使用伪随机负载均衡工具，在保持 $1/\text{poly}(n)$ 失败概率的同时，把随机比特数量压到接近对数级别。

这三者体现了同一技术思想在不同目标下的展开：空间、随机性和失败概率之间并不是孤立的指标，而是可以通过数据结构内部随机化进行统一权衡。

Table 1 三种旋转树结构的主要目标对比

结构	侧重点与结果
基础结构	建立随机旋转框架，实现线性空间和高概率常数时间操作
放大结构	降低失败概率，达到 $O(1/n^{1-\epsilon})$
低随机性结构	减少随机比特，使用量为 $O(\log n \log \log n)$

8 方法评价

该工作最突出的贡献是改变了随机化字典中随机性的承载方式。传统方法将随机性集中在哈希函数中，因而容易受到显式哈希函数族的限制；旋转基数树则将随机性放在节点数组的循环旋转中，从而为负载均衡提供了新的分析对象。

论文的一个启发是，求值时间不是常数的伪随机工具也可能用于常数时间数据结构。低随机性旋转基数树没有在本次查询中直接计算逐渐增强独立性哈希函数，而是用它生成节点初始化所需的随机参数，再利用摊销隐藏初始化成本。

该方法也存在一些局限。首先，分析主要针对 $\Theta(\log n)$ 位键值和相应的 word-RAM 模型，并假设操作序列独立于数据结构使用的随机比特。其次，动态融合节点、辅助树以及多类伪随机工具使整体构造较为复杂。因此，这项工作的主要价值体现在理论界限和设计思路，而不是直接替代工程中的通用哈希表。

9 结论

本文综述了 Kuszmaul 提出的无哈希函数哈希表构造。该工作从旋转基数树出发，将随机性嵌入基数树节点数组的旋转与叠加过程，然后通过增强旋转基数树与低随机性旋转基数树分别推进失败概率和随机比特两个方向的结果。同时，简洁字典转换表明这一思路还能够兼顾接近信息论下界的空间使用。

这篇论文说明，哈希表的概率保证不一定只能通过构造更强的哈希函数来改进。将随机性直接嵌入数据结构，可以形成失败概率、随机比特和空间效率之间新的权衡方式。

参考文献

- [1] KUSZMAUL W. A hash table without hash functions, and how to get the most out of your random bits[C]//2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS). IEEE, 2022: 991-1001.
- [2] PATRASCU M, THORUP M. Dynamic integer sets with optimal rank, select, and predecessor search[C]//2014 IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS). IEEE, 2014: 166-175.
- [3] MCDIARMID C. On the method of bounded differences[J]. Surveys in Combinatorics, 1989, 141: 148-188.
- [4] RAMAN R, RAO S S. Succinct dynamic dictionaries and trees[C]//Automata, Languages and Programming. Springer, 2003: 357-368.